Verified Quantum Circuits: A Lean 4 Workbook

lean-qverify

2026

Contents

Chapter 1 — Why Verify Quantum Circuits? **4**
The basic problem . . . 4
Why simulation doesn't scale . . . 5
Why testing on real hardware is hard . . . 5
What testing can and cannot verify . . . 6
What formal verification gives you . . . 6
Current tools and their limits . . . 7
Why Lean 4 . . . 8
What this library does . . . 8

Chapter 2 — Setup **10**
What you need . . . 10
Step 1: Install elan . . . 10
Step 2: Clone the repository . . . 11
Step 3: Build the library . . . 11
Step 4: Build the book . . . 11
Step 5: Verify a single chapter . . . 12
Step 6: Set up your editor . . . 12
Checking your setup . . . 12
Troubleshooting . . . 12

Chapter 3 — Quantum States **14**
The type `QState n` . . . 14
Basis states . . . 14
Qubit ordering . . . 15
Measurement probabilities . . . 15
Defining your own states . . . 15
What `noncomputable` means . . . 16
Exercises . . . 16

Chapter 4 — Gates **17**
The type `Gate n` . . . 17

Gate matrices . . . 17
Standard matrix identities . . . 18
What the gate type is not . . . 18
Exercises . . . 18

Chapter 5 — Circuits **20**
The type `QCircuit n` . . . 20
Building circuits . . . 20
Gate count and depth . . . 20
Denotational semantics . . . 21
Circuit equivalence . . . 21
Applying a circuit to a state . . . 21
Exercises . . . 22

Chapter 6 — Specifications **23**
What is a specification? . . . 23
The satisfaction relation . . . 23
Writing a spec . . . 24
Named constructors . . . 24
What you can and cannot specify . . . 25
Exercises . . . 25

Chapter 7 — Structural Proofs **26**
What is a structural proof? . . . 26
The main tactic: `simp` . . . 26
CNOT counts . . . 26
Depth proofs . . . 27
RZ-count proofs for the HEA circuit . . . 27
A general pattern . . . 27
Exercises . . . 28

Chapter 8 — Semantic Proofs **29**
What is a semantic proof? . . . 29
The strategy . . . 29
Example: Bell state measurement probability . . . 29
Joint measurement probabilities . . . 30
What these theorems do and don't establish . . . 31
Grover's algorithm . . . 31
When semantic proofs become hard . . . 31
Exercises . . . 32

Chapter 9 — Parametric Circuit Families **33**
The problem with fixed-size circuits . . . 33

`QFamily` and `castSucc` . . . 33
GHZ circuit family . . . 33
Proving properties by induction . . . 34
The Bernstein-Vazirani oracle . . . 34
Why induction is necessary . . . 34
Exercises . . . 35

Chapter 10 — The Python Bridge **36**
Overview . . . 36
Installation . . . 36
Basic usage . . . 37
What `verify_circuit_spec` does . . . 37
Structural vs. semantic verification . . . 37
The `--emit-lean` flag . . . 38
The trust boundary . . . 38
Checking if an angle was recognized . . . 39
Performance . . . 39
Exercises . . . 39

Chapter 11 — Quantum Hoare Logic **40**
What Hoare logic is . . . 40
Density operators . . . 40
Quantum predicates . . . 40
The Hoare triple . . . 41
The four rules . . . 41
Worked example: H gate triple . . . 42
Composing two H gates . . . 43
Build . . . 43

Chapter 12 — ZX-Calculus **44**
What ZX-calculus is . . . 44
Diagram types . . . 44
Spiders . . . 44
Composition and tensor . . . 45
Matrix semantics . . . 45
Fusion rule . . . 45
π-copy rule . . . 46
Gate translation . . . 46
Circuit translation example . . . 47
Build . . . 47

Chapter 1 — Why Verify Quantum Circuits?

The basic problem

A quantum circuit is a program. Like any program, it can be wrong.

Unlike most programs, it is very hard to tell when it is wrong.

Running a classical program and checking its output is usually enough to catch bugs — you compare what you got to what you expected. Quantum programs don't work that way. When you run a quantum circuit, you get a single sample from a probability distribution. The sample is a bit string. You don't get the quantum state, you don't get the amplitudes, and you don't get to look at the circuit mid-run. Measurement collapses the state.

To estimate the probability distribution, you run the circuit hundreds or thousands of times and build a histogram. This is called sampling. It gives you statistical evidence, not certainty.

Here is a concrete example. The Bell state circuit (Hadamard on qubit 0, then CNOT) produces the state $(|00\rangle + |11\rangle)/\sqrt{2}$. Measuring both qubits always gives $|00\rangle$ or $|11\rangle$, with equal probability, and the two qubits are always the same.

Now consider a closely related circuit that inserts a Z gate between the Hadamard and the CNOT (after H, before CNOT). After H, qubit 0 is in $(|0\rangle + |1\rangle)/\sqrt{2}$. Z flips the phase of $|1\rangle$, giving $(|0\rangle - |1\rangle)/\sqrt{2}$. CNOT then produces $(|00\rangle - |11\rangle)/\sqrt{2}$. Measuring both qubits still gives $|00\rangle$ or $|11\rangle$ with equal probability — identical histogram to the first circuit. No finite number of computational-basis measurements can distinguish these two states. They differ only in their relative phase, which is invisible to measurement in the standard basis.

Relative phase matters: algorithms that use Bell states as subroutines, or that measure in a rotated basis, will behave differently depending on which Bell state they actually have. A circuit that produces the wrong Bell state might pass all your tests and still fail inside a larger protocol.

Formal verification gives you "definitely correct" — a mathematical proof, not a probability

estimate.

Why simulation doesn't scale

For small circuits, you can simulate them on a classical computer. A 10-qubit system has 2^10 = 1024 amplitudes. That fits in memory easily. You can compute the exact output state and verify it.

The problem is exponential growth. A 30-qubit system has about a billion amplitudes. Manageable on a modern machine, but slow. A 50-qubit system has $2^{50} \approx 10^{15}$ amplitudes. Storing one complex number per amplitude at 16 bytes each would require about 16 petabytes. That's beyond what any single machine can hold today.

Quantum computers are useful precisely because they operate on circuits at scales that classical simulation cannot reach. The circuits that are hard to simulate classically are also the circuits where direct verification by simulation breaks down.

This doesn't mean classical simulation is useless. You can simulate small versions of your algorithm to check the logic. But you can't simulate the full-size version you actually want to run, and there is no guarantee that bugs won't appear only at scale.

Why testing on real hardware is hard

Real quantum hardware adds another layer of difficulty: noise.

Current quantum processors have error rates on the order of 0.1–1% per gate. A circuit with 100 gates might have a 10–100% chance of introducing at least one error somewhere. This means that even a correct circuit will sometimes produce wrong answers.

From a testing perspective, this creates a problem you don't have in classical computing: you can't tell whether a wrong output came from a bug in your circuit or from hardware noise. Both look the same. You have to run many shots and compare to a noise model, which itself might be wrong.

Hardware access is also limited. Running a large circuit on an IBM Quantum device can mean waiting hours in a queue. Getting enough samples to distinguish a correct circuit from a slightly wrong one can take thousands of shots. Debugging by trial and error on real hardware is slow and expensive.

What testing can and cannot verify

Testing — running a circuit and checking outputs — can tell you some things:

- Whether the circuit syntax is valid (it parses without errors)
- Whether the circuit runs without crashing
- That output probabilities are *roughly* consistent with theory, within statistical noise

Testing cannot tell you:

- Whether output probabilities are *exactly* what the mathematics requires
- Whether entanglement properties are correct (you measure correlations, not entanglement directly)
- Whether the circuit is correct for *all possible* input states (there are 2^n of them; you can't test all of them)
- Whether a property holds for all circuit sizes (e.g., "a GHZ circuit of n qubits has exactly n–1 CNOTs" — testing n=2,3,4 doesn't prove it for all n)

The last point is worth dwelling on. Parametric properties — statements that involve a variable circuit size — are impossible to verify by testing. You can check specific cases, but you cannot check all cases. A formal proof covers all cases at once.

What formal verification gives you

Formal verification means writing a mathematical statement about your circuit and proving it correct.

The statement might be: "This Bell circuit has exactly 2 gates." Or: "Starting from $|00\rangle$, the probability of measuring qubit 0 as 0 is at least 1/2." Or: "For any n, the GHZ circuit of size n uses exactly n+1 gates."

The proof is a sequence of logical steps that a computer checks. If the computer accepts the proof, the statement is true — not probably true, not true within statistical confidence, but mathematically certain given the model.

The computer checking the proof is called a *kernel*. It is a small, carefully audited piece of code whose only job is to verify logical steps. Lean 4's kernel is about 3,000 lines of C++. You don't have to trust the tactics that produce the proof, or the human who wrote it. You only have to trust the kernel.

This is fundamentally different from running the circuit and observing outputs. A proof that "the Bell circuit always produces $|00\rangle$ or $|11\rangle$, never $|01\rangle$ or $|10\rangle$" is a theorem about the ideal circuit model — perfect unitary gates, no noise. It establishes that IF the hardware faithfully implements those gates, the mathematical output is correct. Noise, decoherence,

and gate errors are separate from circuit correctness; formal verification addresses the circuit specification, not the physical implementation.

Current tools and their limits

There are good tools for working with quantum circuits. A few of them do formal verification, but with different scope and emphasis.

Qiskit, Cirq, PennyLane are Python frameworks for building and running circuits. They are useful for simulation, optimization, and hardware access. They have no way to express or check semantic correctness properties. Counting gates in Qiskit is `len(circuit.data)` — that tells you the number of gate objects, not a proved fact about your circuit.

PyZX uses the ZX-calculus to simplify and verify circuit equivalences. It is a genuine verification tool for circuit optimization. Its scope is circuit rewriting under the ZX rules; it does not handle arbitrary user-specified properties or parametric families.

VOQC (Verified Optimizer for Quantum Circuits) is a Coq-based optimizer that produces machine-checked proofs of circuit equivalence for its optimization passes. It is a closer analogue to this project than PyZX, but focuses on compilation correctness rather than user-specified functional specs.

QWIRE and Proto-Quipper are typed quantum programming languages with formal semantics, rooted in linear type theory. They guarantee type-correct programs but do not support arbitrary user-stated theorems about output probabilities or circuit correctness properties.

Quantum Hoare logic is a formal verification method for quantum programs, analogous to Hoare logic for classical programs. It can express pre- and post-conditions over quantum states, but the automation for discharging proof obligations is less mature than in Lean 4.

Q# and Quipper are typed quantum programming languages. Their type systems catch structural errors (wrong qubit types, wrong gate arities). Type checking is weaker than theorem proving — it cannot verify probability bounds or parametric gate counts.

This library combines Lean 4 with Mathlib to get a deep library of complex-number and matrix results with strong automation, plus a Python bridge that makes structural verification accessible from Qiskit workflows without requiring users to write full Lean proofs.

Why Lean 4

Lean 4 is an interactive theorem prover. It is also a practical programming language. You use the same language to write both your programs and your proofs.

What makes it practical for this project:

Mathlib. Lean 4 has a community library called Mathlib with over 150,000 definitions and theorems covering most of undergraduate and graduate mathematics. Matrices, complex numbers, linear algebra, inner product spaces — all of it is already there. We did not have to build the mathematical foundations from scratch.

Automation. Lean 4 has strong tactics for numeric and algebraic reasoning. `norm_num` verifies numeric inequalities. `ring` proves algebraic identities. `omega` handles integer and natural number arithmetic. `simp` applies rewriting rules automatically. Many proofs in this library are one or two lines because these tactics handle the routine steps.

The kernel is small. Lean's kernel is about 3,000 lines of C++. A proof that passes the kernel is trusted. You do not have to trust the tactics, the automation, or the proof search — only the kernel, which is small enough to audit by hand.

A note on alternatives. Coq is a well-established theorem prover with similar logical power to Lean 4. If you already know Coq, you could likely build something similar using it. The main practical differences for this kind of work: Lean 4 has better automation for the numeric and matrix algebra that comes up in quantum mechanics, and Mathlib's matrix theory is more developed than what is available in Coq's standard libraries. Isabelle/HOL is another strong prover with good automation via Sledgehammer; it uses a different type theory and has less natural support for the dependent types that make our circuit model clean. Neither Coq nor Isabelle is a bad choice — Lean 4 was the right tool for this project because Mathlib gave us exactly what we needed.

What this library does

This library gives you:

1. **Lean 4 types for quantum circuits.** `QState n` is a normalized vector in $\mathbb{C}^{(2^n)}$. `Gate n` is an inductive type listing all supported gates for an n-qubit system. `QCircuit n` is a list of gates. These are not simulations — they are mathematical definitions.
2. **A specification language.** `CircuitSpec n` lets you express properties: gate counts, circuit depth, output probabilities, circuit equivalences.
3. **Proved theorems.** 40 theorems about standard circuits — Bell state, GHZ, teleportation, Grover's algorithm, hardware-efficient ansatz, Quantum Hoare Logic rules, and ZX-calculus gate translations — all kernel-checked.

4. **A Python bridge.** From a Qiskit circuit, you can export to OpenQASM 3, parse it into a `QCircuit` term, generate a theorem, and have Lean's kernel check the proof. The verdict comes back to Python.

What the library does not do:

- It does not model noise or decoherence.
- It does not verify quantum error-correcting codes (a separate domain).
- It does not prove quantum algorithms correct in a complexity-theoretic sense.
- It does not automatically discover proofs. You write the proof, or you use the library's existing proved specs.

The rest of this book works through how to use these pieces. Chapters 1–10 cover circuit types, specifications, and the Python bridge. Chapter 11 covers Quantum Hoare Logic — formal pre/post-condition reasoning for quantum programs. Chapter 12 covers ZX-calculus — a graphical rewriting language for circuits with proved matrix semantics.

Chapter 2 — Setup

This chapter gets you from a fresh machine to a working Lean 4 environment with the lean-qverify library compiled and all examples running.

What you need

- A computer running macOS or Linux. Windows works via WSL2.
- About 3 GB of disk space for Lean, Mathlib, and the compiled library.
- An internet connection for the first build (downloading Mathlib's prebuilt cache).

If you already have `elan` installed and Lean 4.14.0 available, skip to the "Clone and build" section.

Step 1: Install elan

`elan` manages Lean toolchain versions, the same way `rustup` manages Rust. You install one version of `elan` and it handles the rest.

```
curl https://raw.githubusercontent.com/leanprover/elan/master/elan-init.sh -sSf | sh
```

When it asks whether to add elan to your PATH, say yes. Then either restart your shell or run:

```
source ~/.elan/env
```

Check it worked:

```
lean --version
# Lean (version 4.14.0, ...)
lake --version
# Lake version 5.0.0-...
```

The exact Lean version is pinned by the `lean-toolchain` file in each project. When you enter a directory with a `lean-toolchain` file, elan switches automatically.

Step 2: Clone the repository

```
git clone https://github.com/akkeshavan/lean-qverify.git
cd lean-qverify
```

The repository has two buildable sub-projects: - The root (`.`) — the library itself - `book/` — this book, which depends on the library

---

Step 3: Build the library

From the repository root:

```
lake update              # resolves Mathlib dependency (~1 min, first run)
lake exe cache get       # downloads prebuilt Mathlib .olean files (~500 MB)
lake build               # compiles the lean-qverify library (~5 min)
```

You should see a long list of compiled files ending with:

```
Build completed successfully.
```

If `lake exe cache get` fails (network restriction, offline), just run `lake build` directly. It will compile Mathlib from source, which takes 30–60 minutes instead of 5.

Step 4: Build the book

From the repository root:

```
cd book
lake update              # resolves the lean-qverify dependency (fast — already downlo
lake exe cache get       # Mathlib cache (fast — same version, already cached)
lake build               # compiles all book examples
```

`lake exe cache get` in `book/` is fast because the Mathlib prebuilt files are already on your machine from step 3. The cache tool checks before downloading.

You should see:

```
Build completed successfully.
```

Step 5: Verify a single chapter

You do not have to build all chapters at once. To check just one:

```
# From inside book/
lake build Book.Ch03.QStates
lake build Book.Ch07.StructuralProofs
```

This builds only the specified module and its dependencies.

Step 6: Set up your editor

For the best experience, use VS Code with the Lean 4 extension. The extension shows types, goals, and errors inline as you type.

```
code .      # open VS Code in the book/ directory
```

Install the **lean4** extension (published by leanprover). It will pick up the Lean version from `lean-toolchain` automatically.

Once the extension loads, open any `.lean` file. Hover over a `#check` to see types. Hover over a tactic step to see the proof state.

Checking your setup

Open `book/Book/Ch02/Setup.lean`. If the file compiles without errors, everything is working.

The file imports the library, checks that the core types exist, and confirms that the 40 proved theorems are accessible. No red underlines means the setup is correct.

Troubleshooting

`lean: command not found`
elan is not on your PATH. Run `source ~/.elan/env` or restart your shell.

`error: unknown package 'lean_qverify'`
Run `lake update` before `lake build`. The manifest has not been generated yet.

`lake exe cache get` prints "no oleans found"
The cache may not exist for your exact platform. Skip this step and let `lake build` compile from source (allow 30–60 minutes).

Build hangs for more than 60 minutes
This is normal if Mathlib is compiling from source. Do not interrupt it. Get some coffee.

Red underlines in VS Code but the file builds on the command line
The Lean extension is still loading the project. It can take 1–2 minutes on first open. Check the spinner in the bottom status bar.

Chapter 3 — Quantum States

The type `QState n`

In this library, an n-qubit quantum state is represented as a structure with two fields:

```
structure QState (n : ℕ) where
  val  : Fin (2 ^ n) → ℂ    -- the amplitude function
  norm : ∑ i, normSq (val i) = 1  -- normalization condition
```

`val` maps each basis index to a complex amplitude. A 1-qubit state has 2 amplitudes. A 2-qubit state has 4. An n-qubit state has 2^n.

The `norm` field is a proof, not a value. Lean's type system enforces that every `QState n` you construct is already normalized. You cannot accidentally create an unnormalized state — the type won't allow it.

Basis states

The library defines the standard basis states for 1 and 2 qubits:

Name	State	Definition
`ket0`	$\|0\rangle$	amplitude 1 at index 0
`ket1`	$\|1\rangle$	amplitude 1 at index 1
`ketPlus`	$\|+\rangle = (\|0\rangle+\|1\rangle)/\sqrt{2}$	equal superposition
`ketMinus`	$\|-\rangle = (\|0\rangle-\|1\rangle)/\sqrt{2}$	equal superposition, negative phase
`ket00`	$\|00\rangle$	amplitude 1 at index 0
`ket01`	$\|01\rangle$	amplitude 1 at index 1
`ket10`	$\|10\rangle$	amplitude 1 at index 2
`ket11`	$\|11\rangle$	amplitude 1 at index 3
`bellPhi`	$(\|00\rangle+\|11\rangle)/\sqrt{2}$	Bell state

Qubit ordering

Index 0 corresponds to the basis state $|00\ldots0\rangle$. Qubit 0 is the *least significant bit* of the index.

So for a 2-qubit state: - Index 0 = $|00\rangle$ (qubit 1 = 0, qubit 0 = 0) - Index 1 = $|01\rangle$ (qubit 1 = 0, qubit 0 = 1) - Index 2 = $|10\rangle$ (qubit 1 = 1, qubit 0 = 0) - Index 3 = $|11\rangle$ (qubit 1 = 1, qubit 0 = 1)

This matches OpenQASM's qubit ordering convention.

Measurement probabilities

The `prob` function gives the probability of measuring a specific basis outcome:

```
def prob {n : ℕ} (ψ : QState n) (k : Fin (2 ^ n)) : ℝ :=
  normSq (ψ.val k)
```

This is the Born rule: the probability of outcome k is $|\text{amplitude_k}|^2$.

Two theorems about `prob` are proved for all states:

```
theorem prob_nonneg : 0 ≤ ψ.prob k         -- probabilities are non-negative
theorem prob_sum_one : ∑ k, ψ.prob k = 1   -- probabilities sum to 1
```

These are proved for *all* `QState n`, not just specific states. They follow from the `norm` field in the structure definition.

Defining your own states

To define a new state, you provide the amplitude function and a proof that the amplitudes are normalized:

```
def myKet : QState 1 :=
  ⟨![1, 0], by simp [Fin.sum_univ_two, Complex.normSq_one]⟩
```

The `![1, 0]` notation creates a vector with those values at indices 0 and 1. The `by simp [...]` block provides the normalization proof.

For states that require real-valued amplitudes like $1/\sqrt{2}$, you use `noncomputable def` because $\sqrt{2}$ is a noncomputable real number:

```
noncomputable def myPlus : QState 1 :=
  ⟨fun _ => (Real.sqrt 2 : ℂ)⁻¹, by
    simp [Fin.sum_univ_two, Complex.normSq_inv, Complex.normSq_ofReal,
          Real.mul_self_sqrt (by norm_num : (2:ℝ) ≥ 0)]⟩
```

What `noncomputable` means

Lean 4 distinguishes computable values (those that can be evaluated to a concrete result) from noncomputable values (those that exist mathematically but cannot be evaluated by a program).

`Real.sqrt 2` is noncomputable because defining it requires classical mathematics (specifically, the least-upper-bound property of real numbers). Lean allows you to prove theorems about noncomputable values — you just can't `#eval` them.

Most quantum states involving irrational amplitudes are noncomputable. This is not a limitation for proving theorems; it's just a distinction in what you can compute vs. what you can reason about.

Exercises

The file `Book/Ch03/QStates.lean` contains the following exercises to run:

1. Check the types of the basis states.
2. Verify that `ket0.prob ⟨0, _⟩ = 1` (measuring $|0\rangle$ in the 0-state gives 1).
3. Verify that `ket0.prob ⟨1, _⟩ = 0` (measuring $|0\rangle$ in the 1-state gives 0).
4. Check that `prob_sum_one` holds for `ket0`.

Build the file:

```
# From inside book/
lake build Book.Ch03.QStates
```

Chapter 4 — Gates

The type `Gate n`

A gate is an operation on an n-qubit system. In this library, gates are an inductive type — a fixed enumeration of all supported operations:

```
inductive Gate (n : ℕ) : Type where
  | H    : Fin n → Gate n         -- Hadamard
  | X    : Fin n → Gate n         -- Pauli X
  | Y    : Fin n → Gate n         -- Pauli Y
  | Z    : Fin n → Gate n         -- Pauli Z
  | S    : Fin n → Gate n         -- S gate
  | T    : Fin n → Gate n         -- T gate
  | RX   : ℝ → Fin n → Gate n    -- rotation around X-axis
  | RY   : ℝ → Fin n → Gate n    -- rotation around Y-axis
  | RZ   : ℝ → Fin n → Gate n    -- rotation around Z-axis
  | CNOT : Fin n → Fin n → Gate n -- controlled-NOT
  | CZ   : Fin n → Fin n → Gate n -- controlled-Z
  | SWAP : Fin n → Fin n → Gate n -- SWAP
  | CCX  : Fin n → Fin n → Fin n → Gate n  -- Toffoli
  | U3   : ℝ → ℝ → ℝ → Fin n → Gate n     -- universal single-qubit
  -- ... and several more
```

The type parameter `n` says how many qubits the system has. The arguments to each constructor say which qubits the gate acts on. For example, `Gate.H ⟨0, _⟩ : Gate 3` is a Hadamard gate on qubit 0 of a 3-qubit system.

Using `Fin n` for qubit indices means out-of-range indices are impossible. `Fin n` is the type of natural numbers less than n. If you try to apply a gate to qubit 5 in a 3-qubit system, Lean will reject your code at compile time.

Gate matrices

Every gate has a corresponding unitary matrix:

```
noncomputable def Gate.matrix {n : ℕ} : Gate n → Matrix (Fin (2^n)) (Fin (2^n)) ℂ
```

This maps a gate to its 2^n × 2^n complex matrix. The matrix acts on the full n-qubit state space via a tensor product construction (single-qubit gates are embedded by tensoring with identity on the other qubits).

The matrices are noncomputable because they involve real-number angles (for RZ, RX, RY) and square roots (for Hadamard). You can prove things about them using Lean's algebraic tactics.

Standard matrix identities

The library proves a number of matrix identities that are used in circuit equivalence proofs. Some examples:

Theorem	Statement
`H_mat_self_mul`	H·H = I
`X_mat_self_mul`	X·X = I
`Rz_mat_add`	RZ(θ)·RZ(ϕ) = RZ(θ+ϕ)
`Rz_zero_id`	RZ(0) = I

These are about the 2×2 matrices themselves. Gate algebra proofs about full circuits lift from these.

What the gate type is not

This representation models ideal gates — perfect unitaries with no noise or decoherence. This is the right model for verifying what a circuit *should* do mathematically. It is not a model of what happens on real hardware, where gates have error rates and coherence times.

Exercises

The file `Book/Ch04/Gates.lean` shows:

1. How to construct gates for a specific system size.
2. That out-of-range qubit indices are rejected by the type system.
3. The `#check` types for gate matrices.
4. A simple matrix identity for H.

Build:

```
lake build Book.Ch04.Gates
```

Chapter 5 — Circuits

The type `QCircuit n`

A circuit is a list of gates:

```
inductive QCircuit (n : ℕ) : Type where
  | empty : QCircuit n
  | cons  : Gate n → QCircuit n → QCircuit n
```

`empty` is the zero-gate circuit. `cons g c` prepends gate `g` to circuit `c`. Gates are applied left to right: the first gate in a `cons` is applied first.

Building circuits

The idiomatic way to write a circuit is using the append operator `++` and the `ofGate` constructor:

```
-- Bell state circuit: H on qubit 0, then CNOT with control 0, target 1
def bellPrep : QCircuit 2 :=
  .cons (.H ⟨0, by norm_num⟩)
  (.cons (.CNOT ⟨0, by norm_num⟩ ⟨1, by norm_num⟩)
  .empty)
```

Or equivalently using `++`:

```
def bellPrep : QCircuit 2 :=
  .ofGate (.H ⟨0, by norm_num⟩) ++
  .ofGate (.CNOT ⟨0, by norm_num⟩ ⟨1, by norm_num⟩)
```

Gate count and depth

Two functions measure circuit structure:

```
def QCircuit.size : QCircuit n → ℕ        -- total number of gates
def countGates : (Gate n → Bool) → QCircuit n → ℕ  -- gates matching a predicate
```

`size` counts all gates. `countGates pred c` counts only the gates where `pred` returns true. This is how you count, say, only CNOT gates:

```
def isCNOT : Gate n → Bool
  | .CNOT _ _ => true
  | _         => false

def cnotCount (c : QCircuit n) : ℕ := countGates isCNOT c
```

Circuit depth (the length of the critical path, accounting for parallelism) is computed by `scheduleDepth`.

Denotational semantics

The `denote` function maps a circuit to its unitary matrix:

```
noncomputable def QCircuit.denote : QCircuit n → Matrix (Fin (2^n)) (Fin (2^n)) ℂ
  | .empty    => 1                          -- identity matrix
  | .cons g c => c.denote * g.matrix        -- compose right to left
```

Gates compose as matrix multiplication. Because `cons g c` applies `g` first, the matrix is `(denote c) * (matrix g)` — the gate-g matrix is on the right, which corresponds to applying it first.

Circuit equivalence

Two circuits are equivalent if they have the same denotation:

```
def QCircuit.equiv (c₁ c₂ : QCircuit n) : Prop :=
  c₁.denote = c₂.denote

notation:50 c₁ " ≅ " c₂ => QCircuit.equiv c₁ c₂
```

$c_1 \cong c_2$ means c_1 and c_2 implement the same unitary transformation. This is the fundamental notion of "these two circuits do the same thing".

Applying a circuit to a state

```
noncomputable def QCircuit.applyTo (c : QCircuit n) (v : Fin (2^n) → ℂ) : Fin (2^n) –
  c.denote.mulVec v
```

`applyTo` multiplies the circuit's unitary matrix by a state vector.

Exercises

`Book/Ch05/Circuits.lean` shows: 1. The Bell circuit definition. 2. Computing its gate count. 3. The type of its denotation matrix. 4. That the Bell circuit is equivalent to itself.

Build:

```
lake build Book.Ch05.Circuits
```

Chapter 6 — Specifications

What is a specification?

A specification is a property you want to prove about a circuit. "This circuit has exactly 2 gates." "The output probability of measuring $|00\rangle$ is at least 1/2." "This circuit is equivalent to the CNOT chain."

In this library, specifications are values of type `CircuitSpec n`. This is an inductive type — a grammar of properties:

```
inductive CircuitSpec (n : ℕ) : Type where
  | implementsMatrix : Matrix ...  → CircuitSpec n   -- exact matrix equality
  | equivalentTo     : QCircuit n  → CircuitSpec n   -- circuit equivalence
  | measurementProb  : QState n → Fin n → Bool → ℝ → CircuitSpec n
  | jointMeasProb    : QState n → Fin (2^n) → ℝ → CircuitSpec n
  | gateCount        : (Gate n → Bool) → ℕ → CircuitSpec n
  | maxDepth         : ℕ → CircuitSpec n
  | both             : CircuitSpec n → CircuitSpec n → CircuitSpec n
```

The satisfaction relation

A circuit `c` satisfies spec `s` is written `c ⊨ s`. The ⊨ symbol is `\models`.

```
noncomputable def satisfies : QCircuit n → CircuitSpec n → Prop
  | .implementsMatrix U       => c.denote = U
  | .equivalentTo c'          => c ≅ c'
  | .measurementProb ψ i b p  => p ≤ measProb c ψ i b
  | .jointMeasProb ψ bs p     => p ≤ normSq ((c.applyTo ψ.val) bs)
  | .gateCount pred k         => countGates pred c ≤ k
  | .maxDepth k               => depth c ≤ k
  | .both s₁ s₂               => satisfies c s₁ ∧ satisfies c s₂
```

This is a Prop — a mathematical statement. `c ⊨ s` is true or false. Proving it true requires a Lean proof.

`gateCount pred k` means the gate count is **at most k** ($\leq$ k), not exactly k. The spec `totalGates 2` is satisfied by any circuit with 2 or fewer gates. To prove an exact count, use a direct equality proof: `show countGates (fun _ => true) myCircuit = 2 from rfl` or `decide` on a concrete circuit. Proving `totalGates 2` twice produces two copies of the same upper-bound statement, not an equality.

Second, `measurementProb` and `jointMeasProb` state **lower bounds** on probabilities (p $\leq$ …). The spec `jointMeasProb` ψ `bs 0` is therefore vacuously satisfied by any circuit, since probability is always non-negative. To prove zero probability, the approach is to compute the output state exactly and show the amplitude is zero.

Writing a spec

Here are some example specs using the named constructors from the library:

```
-- "The circuit uses at most 2 gates total"
def mySpec : CircuitSpec 2 := .gateCount (fun _ => true) 2

-- "The circuit uses at most 1 CNOT"
def cnotSpec : CircuitSpec 2 := maxCNOTs 1

-- "The circuit has depth ≤ 2"
def depthSpec : CircuitSpec 2 := .maxDepth 2

-- "Starting from |00⟩, measuring qubit 0 gives 0 with probability ≥ 1/2"
def probSpec : CircuitSpec 2 :=
  .measurementProb QState.ket00 ⟨0, by norm_num⟩ false (1/2)

-- "Both gate count and depth hold"
def combinedSpec : CircuitSpec 2 := .both (totalGates 2) (.maxDepth 2)
```

Named constructors

The library defines named constructors for common spec patterns:

```
def totalGates (k : ℕ)   : CircuitSpec n := .gateCount (fun _ => true) k
def maxCNOTs   (k : ℕ)   : CircuitSpec n := .gateCount (fun g => g matches .CNOT ..) k
def maxRZGates (k : ℕ)   : CircuitSpec n := .gateCount (fun g => g matches .RZ ..) k
```

What you can and cannot specify

The current spec language covers: - Gate counts (total, by type) - Circuit depth - Output probability lower bounds for single-qubit and joint measurements - Circuit equivalence (same unitary) - Matrix equality

It does not directly support: - Probability upper bounds - Conditional properties ("if the input is X, then the output is Y") - Error rates or noise

For most verification tasks about ideal circuits, the current constructors are enough.

Exercises

`Book/Ch06/Specifications.lean` shows: 1. Writing a spec for the Bell circuit. 2. Writing a spec for the GHZ circuit. 3. Combining specs with `.both`. 4. Checking the types of `satisfies` and ⊨.

Build:

```
lake build Book.Ch06.Specifications
```

Chapter 7 — Structural Proofs

What is a structural proof?

A structural proof is a proof about the circuit structure — gate count, depth, qubit count — rather than what the circuit computes. These proofs do not require knowing the matrix of any gate. They work by counting.

Structural proofs are the easiest to write and the most robust. They don't depend on the exact values of rotation angles. A gate-count proof for a circuit with `RZ(θ)` gates holds for all values of θ simultaneously.

The main tactic: `simp`

Most structural proofs reduce to unfolding definitions and simplifying. The `simp` tactic handles this automatically:

```
theorem bellPrep_twoGates : bellPrep ⊨ totalGates 2 := by
  simp [satisfies, countGates, bellPrep, totalGates]
```

`simp` with the right lemmas reduces `countGates` on the explicit circuit list to a concrete number, then checks it equals 2.

The pattern is always the same: 1. State the theorem: `c ⊨ spec` 2. `simp` with the definitions of `satisfies`, `countGates`, and the circuit

CNOT counts

```
theorem bellPrep_oneCNOT : bellPrep ⊨ maxCNOTs 1 := by
  simp [satisfies, countGates, bellPrep, maxCNOTs]
```

`maxCNOTs k` uses a predicate that returns true only for `CNOT` gates. `simp` reduces the count.

Depth proofs

Depth is slightly more involved. The `scheduleDepth` function computes the length of the critical path. Its definition involves a greedy scheduling algorithm. Proofs need a few more lemmas:

```
theorem bellPrep_depth : bellPrep ⊨ .maxDepth 2 := by
  simp [satisfies, depth, scheduleDepth, bellPrep, gateQubits]
  intro b; fin_cases b <;> decide
```

The `fin_cases b` splits over all possible qubit indices and `decide` checks each case.

RZ-count proofs for the HEA circuit

```
theorem hea4_rzCount : hea4 θ ⊨ maxRZGates 4 := by
  simp [satisfies, countGates, hea4, maxRZGates]
```

Notice that θ is a vector of real-valued angles that can take any value. The gate-count proof holds for all θ simultaneously — the angles don't matter for counting.

A general pattern

When a circuit is parametric (contains angles or is defined by induction on size), structural proofs usually still reduce to `simp`. This works because:

- `countGates` pattern-matches on gate constructors, not their arguments
- `simp` unfolds the list operations definitionally
- `omega` handles any remaining arithmetic

The `ghzCircuit` family (Chapter 9) illustrates this. It is defined by induction on the number of qubits, and its gate-count proof uses exactly this pattern:

```
-- ghzCircuit n has n+1 gates (1 Hadamard + n CNOTs)
theorem ghzCircuit_gateCount (n : ℕ) : ghzCircuit n ⊨ totalGates (n + 1) := by
  induction n with
  | zero =>
    simp [ghzCircuit, satisfies, countGates, totalGates]
  | succ n ih =>
    simp [ghzCircuit, satisfies, countGates, totalGates, castSucc_countGates]
    omega
```

The key extra lemma is `castSucc_countGates`, which says that embedding a circuit into a larger qubit space preserves its gate count. With it, the inductive step reduces to: the count

for `n+1` qubits equals the count for `n` qubits plus 1 (the new CNOT), which `omega` verifies arithmetically.

Exercises

`Book/Ch07/StructuralProofs.lean` proves: 1. `bellPrep ⊨ totalGates 2` 2. `bellPrep ⊨ maxCNOTs 1` 3. `bellPrep ⊨ .maxDepth 2` 4. A combined spec: gate count AND depth together.

Build:

```
lake build Book.Ch07.StructuralProofs
```

Chapter 8 — Semantic Proofs

What is a semantic proof?

A semantic proof is about what the circuit *computes* — the output state or output probabilities — not just its structure.

For example: "The Bell circuit, applied to $|00\rangle$, produces a state where measuring qubit 0 gives 0 with probability at least 1/2."

These proofs require reasoning about the actual unitary matrices and the quantum state they produce. They are harder than structural proofs.

The strategy

Most semantic proofs in this library follow the same two-step pattern:

1. **Compute the output state explicitly.** Compute `circuit.applyTo ket00.val` as a concrete vector of complex amplitudes.
2. **Check the probability numerically.** Use `norm_num` to verify that the absolute-value-squared of the relevant amplitude meets the bound.

Step 1 is the hard part. It requires multiplying out the circuit's unitary matrix, which for a 2-qubit circuit is a 4×4 complex matrix multiplication. The proof writes out the matrix entries and lets `simp` + `norm_num` verify each entry.

Example: Bell state measurement probability

```
private noncomputable def bellOutput : Fin 4 → ℂ :=
  fun k => match k.val with
    | 0 => (Real.sqrt 2 : ℝ)⁻¹
    | 3 => (Real.sqrt 2 : ℝ)⁻¹
    | _ => 0
```

```
theorem bellPrep_applies_to_ket00 :
    bellPrep.applyTo QState.ket00.val = bellOutput := by
  ext k; fin_cases k
  all_goals simp [applyTo, bellPrep, denote, Gate.matrix, QState.ket00, bellOutput, ..
  all_goals norm_num [Real.sqrt_eq_iff_sq_eq, ...]

theorem bellPrep_entangled :
    bellPrep ⊨ .measurementProb QState.ket00 ⟨0, by norm_num⟩ false (1/2) := by
  simp [satisfies, measurementProb, bellPrep_applies_to_ket00, bellOutput, QState.prob
  norm_num [Complex.normSq_inv, Real.mul_self_sqrt]
```

Joint measurement probabilities

The `jointMeasProb` spec checks that a specific basis state has at least a given probability. For the Bell state:

```
-- P(|00⟩) ≥ 1/2 from |00⟩
theorem bellPrep_joint_ket00 :
    bellPrep ⊨ .jointMeasProb QState.ket00 ⟨0, by norm_num⟩ (1/2) := by
  simp only [satisfies]
  rw [bellPrep_applies_to_ket00]
  show (1/2 : ℝ) ≤ normSq (bellOutput ⟨0, by decide⟩)
  simp only [bellOutput, Fin.val]
  norm_num [normSq_ofReal, Real.sq_sqrt (show (0:ℝ) ≤ 2 by norm_num), ...]

-- P(|11⟩) ≥ 1/2 from |00⟩
theorem bellPrep_joint_ket11 :
    bellPrep ⊨ .jointMeasProb QState.ket00 ⟨3, by norm_num⟩ (1/2) := by
  ...  -- same structure
```

A note on the zero-probability theorems: `jointMeasProb ... 0` states a lower bound of 0. Since all probabilities are non-negative, this bound holds vacuously for any circuit. The theorem named `bellPrep_joint_ket01_zero` proves `0 ≤ normSq (...)`, which by itself is trivially true. What gives it meaning is that the proof goes through `bellPrep_applies_to_ket00`, which establishes the exact output amplitude at index 1 is zero. The real content is in `bellPrep_applies_to_ket00` — the zero theorems just package the consequence in spec form.

To prove zero probability non-vacuously, the direct approach is:

```
-- This directly computes the exact amplitude, proving P(|01⟩) = 0:
example : Complex.normSq (bellPrep.applyTo QState.ket00.val ⟨1, by decide⟩) = 0 := by
  rw [bellPrep_applies_to_ket00]
```

```
  simp [bellOutput, Fin.val]
```

What these theorems do and don't establish

The `bellPrep_joint_ket00` and `bellPrep_joint_ket11` theorems, together with `prob_sum_one`, imply that both outcomes have probability exactly 1/2.

What they do not establish: the **relative phase** between the $|00\rangle$ and $|11\rangle$ components. The states $(|00\rangle + |11\rangle)/\sqrt{2}$ and $(|00\rangle - |11\rangle)/\sqrt{2}$ have identical computational-basis probabilities. Distinguishing them requires reasoning about the exact output amplitudes, which `bellPrep_applies_to_ket00` does provide — but only through the exact vector equality, not through the probability lower-bound specs.

Similarly, `bellPrep_entangled` and `bellPrep_entangled_one` prove that each single-qubit marginal is at least 1/2. That is a correct and kernel-checked statement, but it is a measurement probability bound — it does not directly prove entanglement in the information-theoretic sense, since a separable mixed state could satisfy the same marginal bounds. The proof establishes the ideal output distribution, not entanglement as a formal property.

Grover's algorithm

The library includes a proof of one step of Grover's search on 2 qubits:

```
theorem groverStep2_correct :
    groverStep2 ⊨ .measurementProb groverInput ⟨0, by norm_num⟩ true (99/100) := by
  ...
```

This is a 4×4 matrix computation verified by `norm_num`. The proof is about 30 lines of explicit matrix arithmetic.

When semantic proofs become hard

For large circuits, explicit matrix computation becomes unwieldy. A 5-qubit circuit has a 32×32 matrix. Writing out 1024 entries for the proof is not practical.

For large circuits, the better strategy is usually: - Decompose the circuit into verified subcircuits - Use proved equivalences to simplify - State weaker specs that are easier to verify (lower probability bounds, partial structural properties)

The library's existing theorems serve as a foundation you can build on.

Exercises

`Book/Ch08/SemanticProofs.lean` contains: 1. The Bell output state computation. 2. Checking one Bell measurement probability. 3. A joint probability proof for $|00\rangle$.

Build:

```
lake build Book.Ch08.SemanticProofs
```

Chapter 9 — Parametric Circuit Families

The problem with fixed-size circuits

All the circuits in the previous chapters have a fixed qubit count: 2 for Bell, 2 for Grover, 4 for HEA. Their `QCircuit n` types have a concrete `n`.

Many interesting circuits are defined for any number of qubits. A GHZ state circuit takes any `n` and produces the n-qubit GHZ state. A Bernstein-Vazirani oracle takes any `n` and a secret string of length `n`.

To reason about these circuits, you need induction. And to write induction over circuits, you need a way to build n-qubit circuits from (n−1)-qubit circuits.

`QFamily` and `castSucc`

`QFamily f` is a family of circuits parameterized by qubit count:

```
def QFamily (f : ℕ → ℕ) := (n : ℕ) → QCircuit (f n)
```

`QCircuit.castSucc` embeds an n-qubit circuit into an (n+1)-qubit space by leaving the new qubit untouched:

```
def QCircuit.castSucc : QCircuit n → QCircuit (n+1)
```

This lets you define n-qubit circuits inductively: start from a base case, then at each step embed the previous circuit into a larger space and append new gates.

GHZ circuit family

The GHZ state for n+1 qubits is defined inductively:

```
def ghzCircuit : (n : ℕ) → QCircuit (n + 1)
  | 0     => .ofGate (.H ⟨0, by norm_num⟩)
```

```
  | n + 1 => (ghzCircuit n).castSsucc ++
             .ofGate (.CNOT ⟨n, by omega⟩ ⟨n+1, by omega⟩)
```

Base case: a single Hadamard. Inductive step: embed the previous circuit and add one CNOT.

Proving properties by induction

```
theorem ghzCircuit_gateCount (n : ℕ) :
    ghzCircuit n ⊨ totalGates (n + 1) := by
  induction n with
  | zero   => simp [ghzCircuit, satisfies, countGates, totalGates]
  | succ n ih =>
    simp [ghzCircuit, satisfies, countGates, totalGates, castSucc_countGates]
    omega
```

`castSucc_countGates` is a library lemma that says gate counts are preserved under `castSucc`. With it, the inductive step reduces to arithmetic that `omega` handles.

The Bernstein-Vazirani oracle

The BV oracle circuit for an n-bit secret string `s` uses at most n CNOTs:

```
theorem bvOracle_cnotCount_le (n : ℕ) (s : Fin n → Bool) :
    bvOracle n s ⊨ maxCNOTs n := by
  ...
```

This is proved by induction on `n`. At each step, the oracle for `s` on `n+1` qubits either adds one CNOT (if s(n) = true) or adds nothing (if s(n) = false). Either way, the CNOT count stays $\leq$ n+1.

Why induction is necessary

You cannot replace these proofs with a fixed finite check. There is no largest n. The theorem `ghzCircuit_gateCount (n : ℕ)` holds for all natural numbers n — including n = 1000 or n = 10^9. Testing n = 2, 3, 4 does not establish this.

Lean's induction principle is what makes "for all n" statements possible. The proof for the `succ n` case uses the proof for `n` as a hypothesis. The chain goes back to the base case, which is concrete.

Exercises

`Book/Ch09/ParametricFamilies.lean` shows: 1. The `ghzCircuit` definition. 2. `ghzCircuit_gateCount` proof. 3. The BV oracle definition. 4. The `bvOracle_cnotCount_le` statement.

Build:

```
lake build Book.Ch09.ParametricFamilies
```

Chapter 10 — The Python Bridge

Overview

The previous chapters worked entirely inside Lean. In practice, most quantum programmers write circuits in Python using Qiskit, Cirq, or a similar framework. This chapter covers the bridge from Python to Lean.

The pipeline is:

```
Qiskit circuit
    ↓  qiskit.qasm3.dumps(circuit)
OpenQASM 3 string
    ↓  lean_qverify parser
QCircuit n term
    ↓  write theorem + proof
lake env lean
    ↓  kernel check
VerifyResult
```

Each step is handled by the `lean_qverify` Python package.

Installation

```
pip install lean-qverify  # or: pip install -e . from the repo root
```

Or just import directly from the repo:

```
import sys
sys.path.insert(0, "/path/to/lean-qverify")
from lean_qverify import verify_circuit_spec
```

Basic usage

```
from qiskit import QuantumCircuit
from lean_qverify import verify_circuit_spec, CircuitSpec

# Define a Qiskit circuit
qc = QuantumCircuit(2)
qc.h(0)
qc.cx(0, 1)

# Define a structural spec
spec = CircuitSpec.gate_count(total=2)

# Verify it
result = verify_circuit_spec(qc, spec)
print(result.verdict)   # STRUCT_PROVED
print(result.kernel_checked)  # True
```

What `verify_circuit_spec` does

1. Calls `qiskit.qasm3.dumps(qc)` to get an OpenQASM 3 string.
2. Parses the QASM into a `QCircuit n` term using the Lean parser.
3. Generates a `.lean` file containing the circuit definition and the theorem.
4. Calls `lake env lean` to kernel-check the proof.
5. Returns a `VerifyResult` with the verdict.

The `kernel_checked` field on the result tells you whether Lean's kernel accepted the proof. A result with `kernel_checked = True` and `verdict = STRUCT_PROVED` means the property was mathematically proved.

Structural vs. semantic verification

Structural verification (gate counts, depth) is fast and fully automatic. You specify the property, the bridge generates the proof, Lean checks it. No user-written Lean required.

Semantic verification (output probabilities) requires a hand-written Lean proof. The bridge can generate the circuit term and the theorem statement; the proof of the theorem is something you write.

```
from lean_qverify import verify_circuit_spec, CircuitSpec

# Structural: automatic
```

```
result = verify_circuit_spec(qc, CircuitSpec.gate_count(total=2))

# Semantic: bridge emits the theorem, you prove it in Lean
result = verify_circuit_spec(qc, CircuitSpec.measurement_prob(qubit=0, outcome=0, boun
# result.verdict = PARSED  (not STRUCT_PROVED — needs a hand proof)
```

The `--emit-lean` flag

The verifier binary has an `--emit-lean` mode that writes the circuit as a Lean definition:

```
lean-qverify-check circuit.qasm --emit-lean
```

Output:

```
noncomputable def generatedCircuit : QCircuit 2 :=
  .cons (.H ⟨0, by norm_num⟩)
  (.cons (.CNOT ⟨0, by norm_num⟩ ⟨1, by norm_num⟩)
  .empty)
```

This is the starting point for writing your own Lean proofs about a Qiskit-generated circuit.

The trust boundary

When the bridge reports `STRUCT_PROVED` and `kernel_checked = True`, it means Lean's kernel accepted the proof. The trusted computing base is:

1. **Lean's kernel** (~3,000 lines of C++) — this is what you are trusting to check logical steps correctly.
2. **The QASM parser** — the translation from the QASM string to a `QCircuit n` term. If the parser incorrectly translates a gate (wrong qubit index, wrong angle, dropped gate), the proof will be about a different circuit than the one you intended. Parser correctness is established through `checkWellFormed_sound` and the concrete gate-translation theorems, but is not automatically checked for every circuit.
3. **The angle recognizer** — floating-point angles from QASM are matched against known rational multiples of π. An unrecognized angle produces a warning; the circuit is still parsed but semantic proofs will not apply.

The upshot: a `STRUCT_PROVED` verdict means the structural property (gate count, depth) holds for the Lean `QCircuit` term that the parser produced from your QASM. If the parser is faithful to the QASM, the property holds for your Qiskit circuit.

The QASM subset supported by the parser covers the gates used in the examples throughout this book. Classical control, measurement operations, and custom gate definitions are outside the current parser scope.

Checking if an angle was recognized

The parser converts floating-point angles from QASM to exact real-number expressions in Lean. For example, `rz(pi/2)` becomes `Real.pi / 2`. This is necessary for the proof to go through — `norm_num` works with exact expressions, not floating-point approximations.

The parser recognizes over 30 rational multiples of π. If your circuit uses an angle outside this set, the parser returns a warning and the semantic proof will not apply. Structural proofs (gate counts, depth) are unaffected by angles.

```
result = verify_circuit_spec(qc, spec)
if result.warnings:
    print(result.warnings)  # lists any unrecognized angles
```

Performance

The main cost is Lean startup time: about 10–15 seconds per verification call. This is Lean loading the compiled library, not proof search time. Once loaded, proof checking is fast (milliseconds).

If you are verifying many circuits, reuse the same Lean process by keeping the bridge alive rather than spawning a new one per call.

Exercises

`Book/Ch10/PythonBridge.lean` shows: 1. The QASM program structure that the parser produces. 2. The well-formedness theorem for a Bell circuit QASM program. 3. How to inspect the `QASMProgram` type.

There is also a Python script `Book/Ch10/example.py` that runs a full end-to-end verification using the bridge. Run it from the repo root:

```
python book/Book/Ch10/example.py
```

Build the Lean part:

```
lake build Book.Ch10.PythonBridge
```

Chapter 11 — Quantum Hoare Logic

What Hoare logic is

Classical Hoare logic reasons about programs with triples `{P} c {Q}`. P is true before running c, Q is true after. If P holds and you run c, Q holds.

For quantum programs this needs to change. Quantum states are not classical values. Instead of "variable x equals 5," preconditions and postconditions are measurement probabilities.

Density operators

A quantum state ρ is a density operator: a 2^n × 2^n complex matrix satisfying: - Hermitian: $\rho = \rho\dagger$ - Positive semidefinite: $\langle\psi|\rho|\psi\rangle \geq 0$ for all $|\psi\rangle$ - Unit trace: $\mathrm{tr}(\rho) = 1$

For a pure state $|\psi\rangle$, $\rho = |\psi\rangle\langle\psi|$.

A circuit c acts on ρ by unitary conjugation:

`c(ρ) = c.denote · ρ · c.denote`†

Quantum predicates

A quantum predicate `QPred n` is a 2^n × 2^n Hermitian matrix A with $0 \leq A \leq I$ in the Löwner order. The number $\mathrm{tr}(A \cdot \rho)$ is the probability that a measurement corresponding to A succeeds when the system is in state ρ.

Two concrete predicates:

- `QPred.trivial n`: mat = 0. $\mathrm{tr}(0 \cdot \rho) = 0$. Never observed, regardless of ρ.
- `QPred.full n`: mat = I. $\mathrm{tr}(I \cdot \rho) = \mathrm{tr}(\rho) = 1$. Always observed for unit-trace ρ.

These are useful as boundary cases when building derivations.

The Hoare triple

`HoareTriple A c B` means: for all density operators ρ with $\mathrm{tr}(\rho) = 1$,

`tr(A · ρ) ≤ tr(B · c(ρ))`

If A is satisfied with probability p before running c, then B is satisfied with at least probability p after.

In Lean:

```
#check @HoareTriple
-- HoareTriple : QPred n → QCircuit n → QPred n → Prop
```

The four rules

Skip rule

The empty circuit doesn't change the state, so any predicate A is preserved:

```
theorem hoare_skip (A : QPred n) : HoareTriple A .empty A
```

Example:

```
theorem ex_skip (A : QPred 1) : HoareTriple A .empty A :=
  hoare_skip A
```

Gate rule

For a gate g with unitary matrix U satisfying U†U = I, applying g transforms the precondition by Löwner conjugation:

```
theorem hoare_gate (A : QPred n) (g : Gate n)
    (hU : g.matrix.conjTranspose * g.matrix = 1) :
    HoareTriple (A.conj g.matrix hU) (ofGate g) A
```

`A.conj U hU` is the predicate U† · A · U. The gate rule says: if U† · A · U holds before the gate, then A holds after.

Sequential rule

If c_1 transforms predicate A into B, and c_2 transforms B into C, then c_1 followed by c_2 transforms A into C. Requires c_1 to be unitary:

```
theorem hoare_seq
    (hU₁ : c₁.denote.conjTranspose * c₁.denote = 1)
    (h₁ : HoareTriple A c₁ B) (h₂ : HoareTriple B c₂ C) :
    HoareTriple A (c₁ ++ c₂) C
```

Consequence rule

If A' $\leq$ A (i.e., A $-$ A' is PSD) and B $\leq$ B' (i.e., B' $-$ B is PSD), then a triple for A/B implies one for A'/B':

```
theorem hoare_consequence
    (hA : matPSD (A.mat - A'.mat)) (h : HoareTriple A c B)
    (hB : matPSD (B'.mat - B.mat)) :
    HoareTriple A' c B'
```

Use this to weaken a precondition or weaken a postcondition. Under the expectation interpretation, a larger QPred B' (B' $\geq$ B in Löwner order) has a higher probability tr(B'$\cdot\rho$') — it is less demanding, not more. Both directions weaken.

Worked example: H gate triple

The H gate is its own adjoint (H† = H) and unitary (H $\cdot$ H = I).

Steps:

1. State the unitarity condition for the single-qubit H gate:

```
private lemma H_gate1_unitary :
    (Gate.H (⟨0, by norm_num⟩ : Fin 1)).matrix.conjTranspose *
    (Gate.H (⟨0, by norm_num⟩ : Fin 1)).matrix = 1 := by
  simp only [Gate.matrix, embedGate1_one_qubit]
  rw [H_mat_conjTranspose, H_mat_unitary]
```

   - `simp only [Gate.matrix, embedGate1_one_qubit]` reduces both sides to `H_mat† * H_mat`.
   - `rw [H_mat_conjTranspose]` replaces `H_mat†` with `H_mat` (H is Hermitian).
   - `rw [H_mat_unitary]` closes `H_mat * H_mat = 1`.
   - Note: the two-step approach is needed because `H_mat_conjTranspose` rewrites across a `Fin 2` / `Fin (2^1)` type boundary that `simp only` cannot bridge in one

pass.

2. Apply the gate rule:

```
theorem H_gate_triple (A : QPred 1) :
    HoareTriple (A.conj _ H_gate1_unitary)
                (ofGate (.H ⟨0, by norm_num⟩)) A :=
  hoare_gate A (.H ⟨0, by norm_num⟩) H_gate1_unitary
```

The precondition is H† · A · H = H · A · H. After the gate, A holds.

Composing two H gates

Applying H twice returns to the original state (H · H = I).

Using `hoare_seq`: `- h₁ : HoareTriple (H·A·H) H A` from `H_gate_triple A` - `h₂ : HoareTriple (H·(H·A·H)·H) H (H·A·H)` from `H_gate_triple (A.conj H _)` - `hoare_seq hU h₂ h₁ : HoareTriple (H·(H·A·H)·H) (H;H) A`

The precondition `H · (H · A · H) · H` simplifies to `(H·H)·A·(H·H) = I·A·I = A`, using `H_mat_unitary` and associativity. Formalizing this simplification uses the consequence rule with `A' = A` and `matPSD ((H·(H·A·H)·H).mat − A.mat) = matPSD 0 = trivial`.

Build

From inside `book/`:

```
lake build Book.Ch11.QuantumHoareLogic
```

Chapter 12 — ZX-Calculus

What ZX-calculus is

ZX-calculus is a graphical language for quantum computation. Quantum circuits are drawn as diagrams. Two diagrams are equivalent when their matrices are equal — you can rewrite one into another without changing the computation.

This library formalizes ZX-calculus in Lean and proves matrix semantics for all standard diagrams.

Diagram types

A `ZXDiagram m n` has m input wires and n output wires. Its matrix semantics is:

`ZX⟦d⟧ : Matrix (Fin (2^n)) (Fin (2^m)) ℂ`

Two primitive diagrams: `- wire : ZXDiagram 1 1` — identity wire; `ZX⟦wire⟧ = I` - `hadamard : ZXDiagram 1 1` — Hadamard gate; `ZX⟦hadamard⟧ = H_mat`

Spiders

Spiders are multi-legged nodes, each with an angle parameter α.

Z-spider `zSpider m n` α `: ZXDiagram m n`

A "green" node with m inputs and n outputs. For 1-in-1-out Z-spiders, the angle selects a phase gate:

Angle	Matrix
π	Z gate (Z_mat)
$\pi/2$	S gate (S_mat)

Angle	Matrix
$\pi/4$	T gate (T_mat)

X-spider `xSpider m n` α `: ZXDiagram m n`

A "red" node. Its matrix is the Hadamard conjugate of the Z-spider's matrix. For 1-in-1-out: - $\alpha = \pi$: X gate (X_mat)

Composition and tensor

- d_1 `⊚` d_2 — d_1 then d_2 sequentially. Semantics: `ZX⟦`d_1 `⊚` d_2`⟧ = ZX⟦`d_2`⟧ ∗ ZX⟦`d_1`⟧`.
- d_1 `↑⊗` d_2 — d_1 and d_2 side by side. Semantics: Kronecker product.

Matrix semantics

`ZX⟦d⟧` interprets each diagram constructor. Basic equations:

```
ZX⟦wire⟧     = 1      -- identity
ZX⟦hadamard⟧ = H_mat  -- Hadamard gate
```

Z-spider phase gates verified in Lean:

```
example : ZX⟦zSpider 1 1 Real.pi⟧ = Z_mat := by
  simp [denote_zSpider, zSpiderMat_11_pi]

example : ZX⟦zSpider 1 1 (Real.pi / 2)⟧ = S_mat := by
  simp [denote_zSpider, zSpiderMat_11_half_pi]

example : ZX⟦zSpider 1 1 (Real.pi / 4)⟧ = T_mat := by
  simp [denote_zSpider, zSpiderMat_11_quarter_pi]
```

Fusion rule

Two X-spiders connected in series fuse into one with phases added:

`ZX⟦xSpider m k` α `⊚ xSpider k n` β`⟧ = ZX⟦xSpider m n (`α `+` β`)⟧`

This holds for all m, k, n ≥ 1. Fusion is the core ZX simplification: a chain of same-colored spiders collapses into one.

In Lean:

```
example (m n : ℕ) (hm : 0 < m) (hn : 0 < n) (α β : ℝ) :
    ZX⟦(xSpider m 1 α) ⊚ (xSpider 1 n β)⟧ = ZX⟦xSpider m n (α + β)⟧ :=
  xSpider_fusion m 1 n hm (by norm_num) hn α β
```

π-copy rule

A Z(π) node commutes through a forking X-spider by duplicating onto both outputs:

```
ZX⟦zSpider 1 1 π ⊚ xSpider 1 2 0⟧
  = ZX⟦xSpider 1 2 0 ⊚ (zSpider 1 1 π ↑⊗ zSpider 1 1 π)⟧
```

In Lean, this is proved as:

```
theorem pi_copy :
    ZX⟦(zSpider 1 1 Real.pi) ⊚ (xSpider 1 2 0)⟧ =
    ZX⟦(xSpider 1 2 0) ⊚ ((zSpider 1 1 Real.pi) ↑⊗ (zSpider 1 1 Real.pi))⟧
```

The proof works by computing both sides as concrete 4×4 matrices and showing they are equal.

Gate translation

`gateToZX`$_1$ `: Gate 1 → ZXDiagram 1 1` maps standard 1-qubit gates to ZX diagrams:

Gate	ZX diagram
H	`hadamard`
X	`xSpider 1 1 π`
Z	`zSpider 1 1 π`
S	`zSpider 1 1 (π/2)`
T	`zSpider 1 1 (π/4)`

The soundness theorem:

```
theorem gateToZX_1_sound (g : Gate 1) (hg : isStdGate_1 g) :
    ZX⟦gateToZX_1 g⟧ = g.matrix
```

`isStdGate₁` is True for H/X/Z/S/T and False for other gates (parametric and controlled gates are not yet supported).

Single gate examples:

```
-- H maps to hadamard:
example : ZX⟦gateToZX₁ (.H ⟨0, by norm_num⟩)⟧ = (Gate.H ⟨0, by norm_num⟩).matrix :=
  gateToZX₁_sound _ trivial

-- Z maps to zSpider 1 1 π:
example : ZX⟦gateToZX₁ (.Z ⟨0, by norm_num⟩)⟧ = (Gate.Z ⟨0, by norm_num⟩).matrix :=
  gateToZX₁_sound _ trivial
```

Circuit translation example

`circuitToZX₁ : QCircuit 1 → ZXDiagram 1 1` folds gate translation over a circuit. `isStdCircuit₁` checks that every gate is in the supported set.

The circuit [H; Z]:

```
noncomputable def hzCircuit : QCircuit 1 :=
  .cons (.H ⟨0, by norm_num⟩) (.cons (.Z ⟨0, by norm_num⟩) .empty)

theorem hz_circuit_zx : ZX⟦circuitToZX₁ hzCircuit⟧ = hzCircuit.denote :=
  circuitToZX₁_sound hzCircuit ⟨trivial, trivial, trivial⟩
```

`⟨trivial, trivial, trivial⟩` is the proof that `isStdCircuit₁ hzCircuit`: the three components are `isStdGate₁ H = True`, `isStdGate₁ Z = True`, and `isStdCircuit₁ empty = True`.

Build

From inside `book/`:

```
lake build Book.Ch12.ZXCalculus
```